Project Idea: Vital Studio – Staff and Task Management Dashboard using ASP.NET Core and C#

Your idea for **Vital Studio** is to develop a centralized management system that allows managers to monitor employee performance, supervise teams, assign tasks to supervisors, and communicate important work-related information from one dashboard.

Using **ASP.NET Core with C#** for the backend and a modern web interface for the frontend is a suitable approach. You can build the system as a web application with a database that stores staff information, teams, tasks, task statuses, and internal messages.

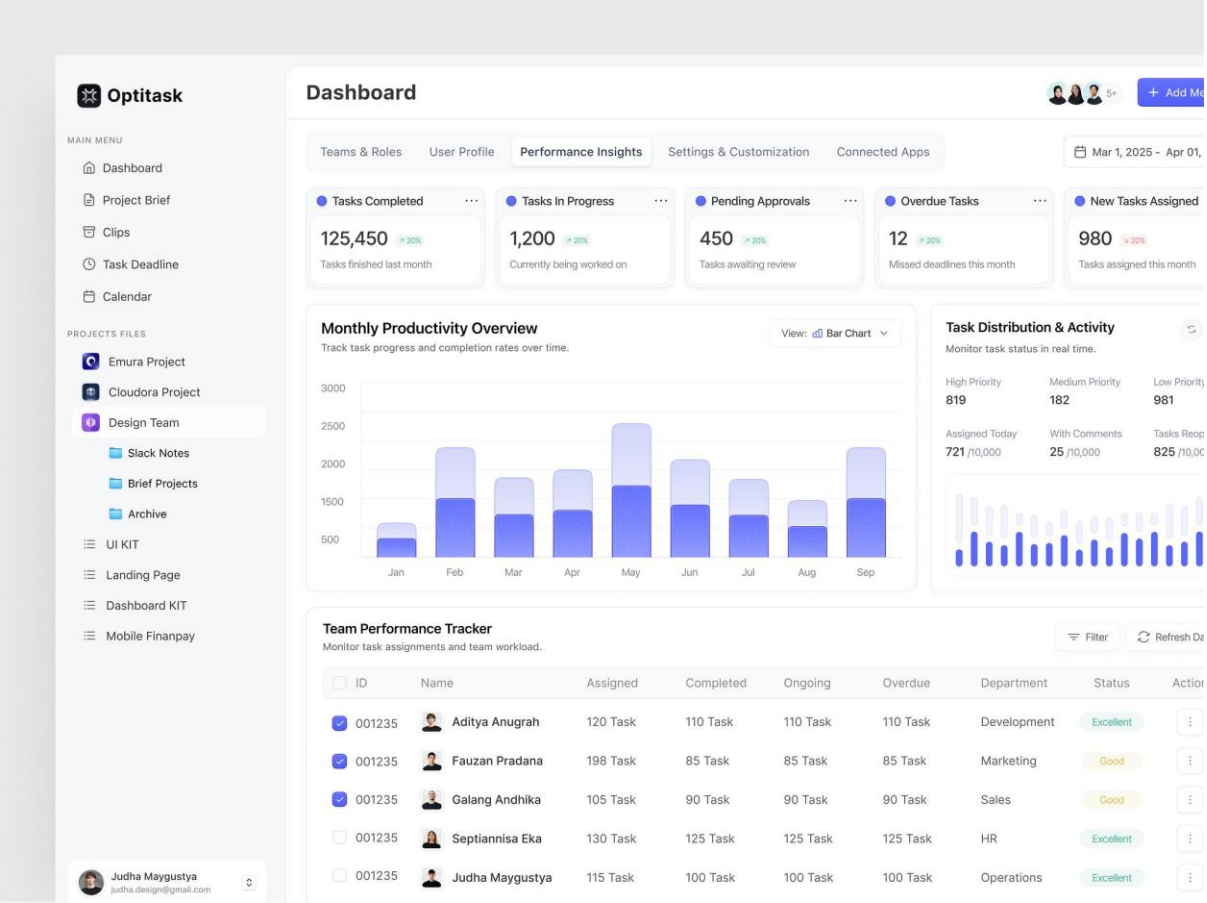
1. Main objective of the project

The primary objective is to provide management with a comprehensive overview of daily operations, including:

- Monitoring work performed by employees and supervisors.
- Tracking tasks assigned to each employee according to their team.
- Viewing task statuses and completion rates.
- Assigning tasks to supervisors based on their teams and areas of responsibility.
- Sending announcements, instructions, and administrative updates.
- Generating reports that help managers identify delays, workload imbalances, and operational performance.

The dashboard should help managers answer questions such as: *Who is working on each task? Which teams are performing well? Which tasks are delayed? What work has been completed?*

2. Recommended dashboard design



The manager's dashboard should be organized into a few main sections so that the most important information is visible immediately.

1. Manager Overview

A summary of staff, tasks, progress, and overdue work.

2. Staff and Team Performance

Compare employee workloads, completed tasks, and performance by team.

3. Task Management

Create, assign, edit, monitor, and review tasks.

4. Communication Center

Send announcements and targeted messages to supervisors, teams, or all staff.

5. Reports and Analytics

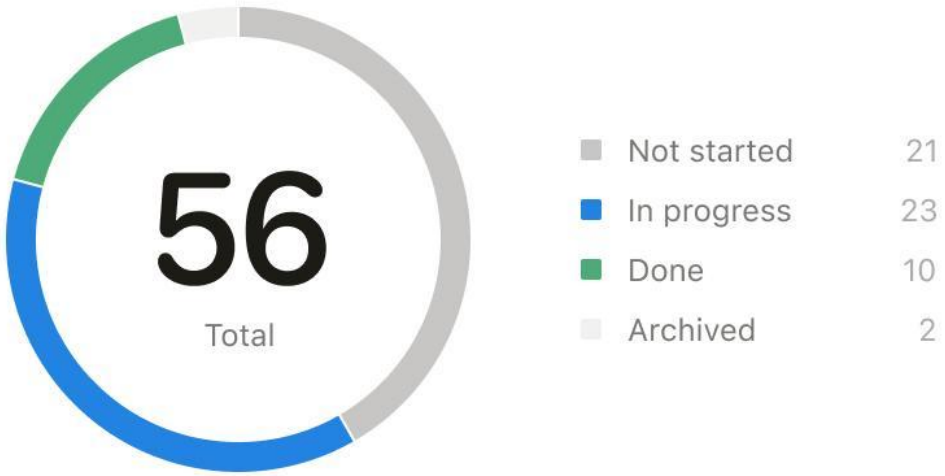
Review historical performance, completion rates, and overdue tasks.

3. Dashboard charts and visualizations

The most important improvement is to make the dashboard interactive and data-driven, rather than displaying only static numbers.

A. Task status overview

A doughnut chart showing the distribution of all tasks.

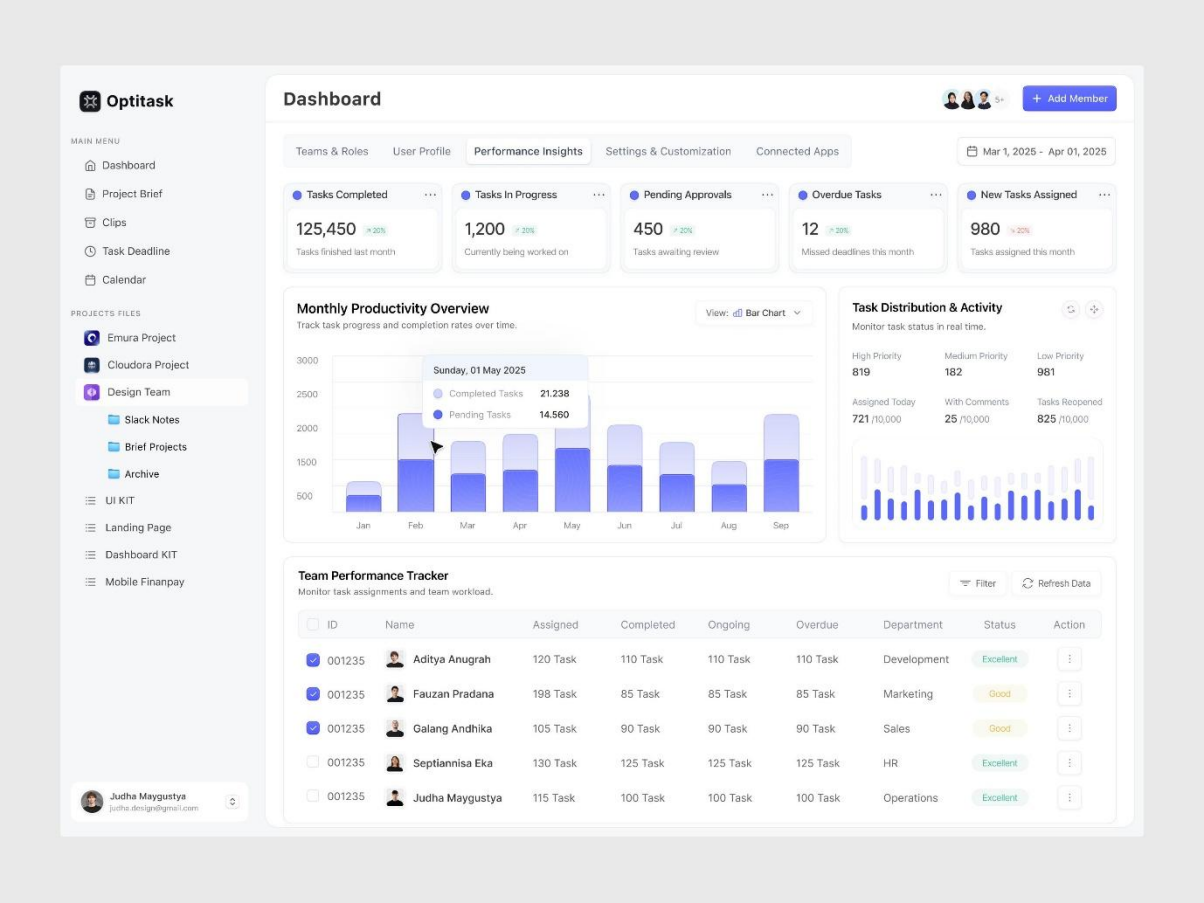


Use the following categories:

- Received
- In Progress
- Completed
- Incomplete
- New
- Deleted (preferably tracked separately from active tasks)

B. Employee performance by team

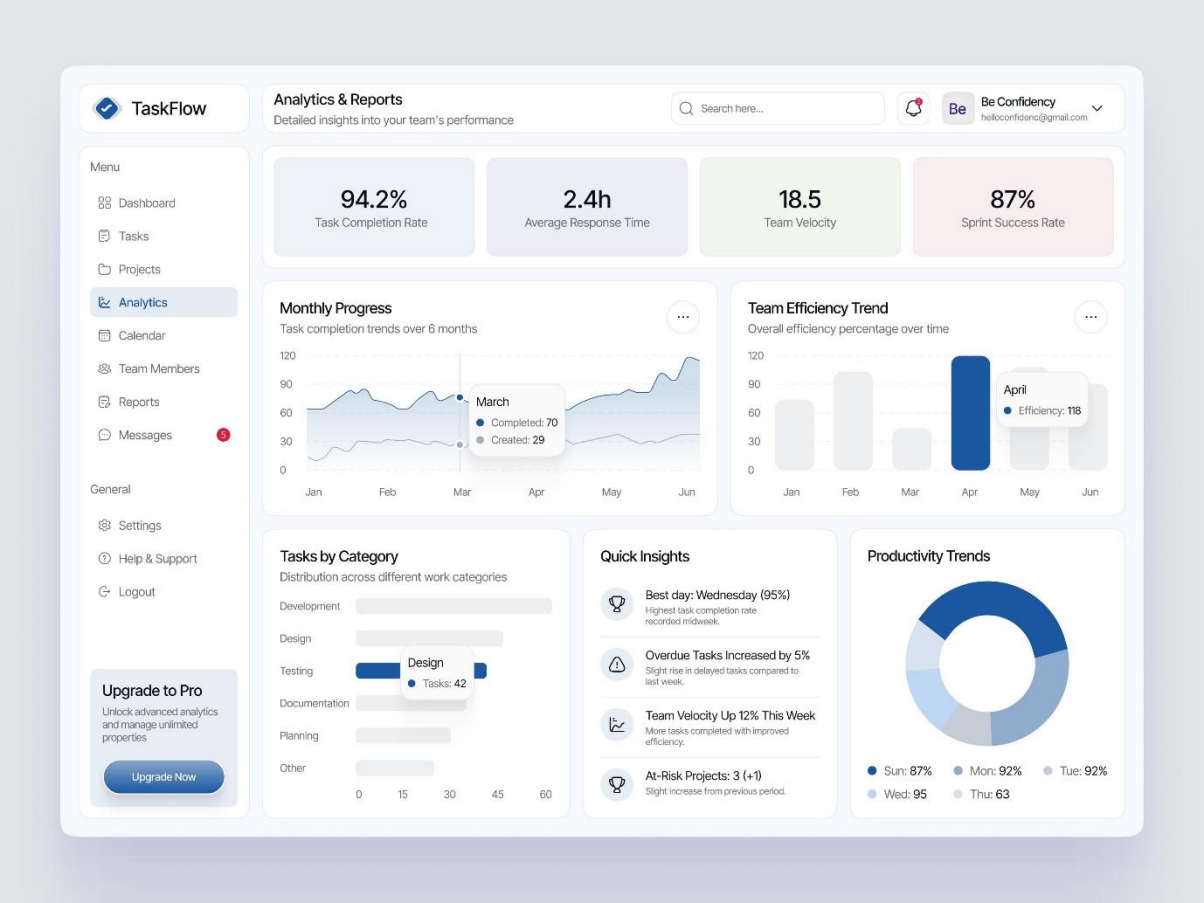
A grouped bar chart comparing assigned, completed, and outstanding tasks.



Managers should be able to filter the chart by team, employee, supervisor, and date range.

C. Task completion over time

A line chart showing whether teams are completing work on schedule.



Use daily, weekly, and monthly views to identify improvements or delays.

Suggested summary cards

At the top of the dashboard, display key performance indicators (KPIs).

Dashboard card	**What it measures**
Total Staff	Active employees and supervisors
Total Tasks	Tasks created within the selected scope
New / Received	Tasks awaiting review or acknowledgment
In Progress	Tasks currently being worked on
Completed	Tasks successfully finished

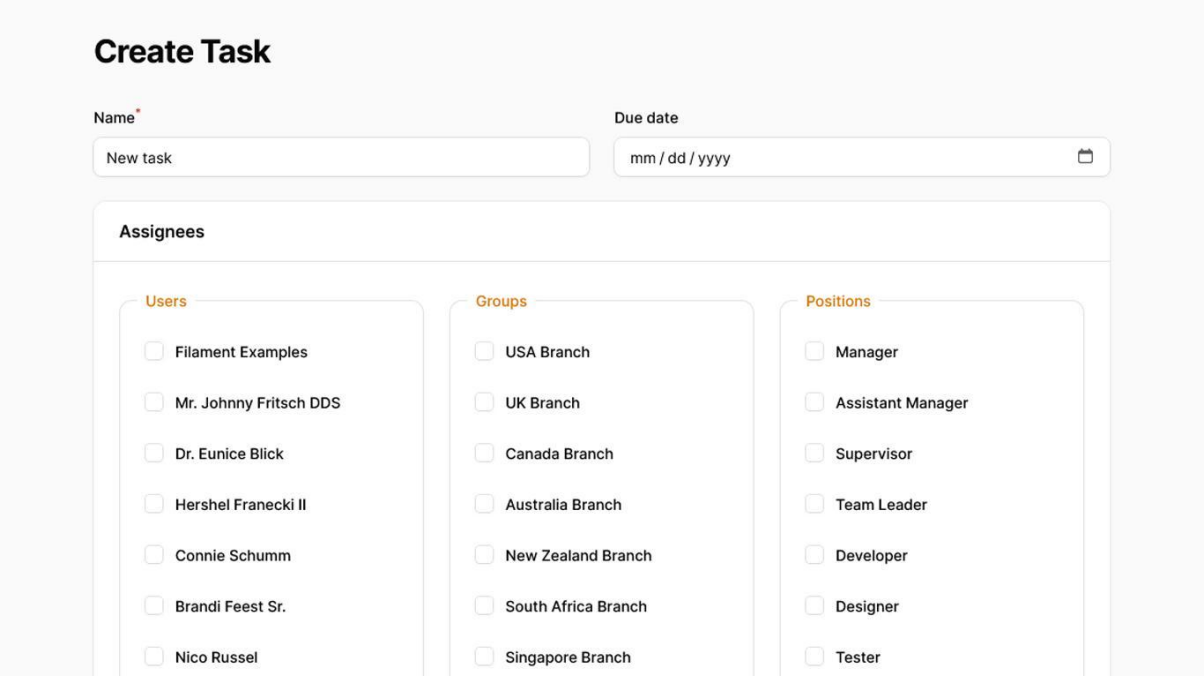
Overdue	Tasks past their due dates that remain unfinished
Completion Rate	Completed tasks divided by eligible assigned tasks

The numbers and charts should come directly from the database. Selecting a chart segment or summary card should open a filtered list of the underlying tasks.

Important: Define the difference between *New* and *Received*. For example, *New* can mean a task has just been created, while *Received* means the assignee has acknowledged it. Similarly, an overdue task is usually better represented as a due-date condition in addition to its workflow status.

4. Task assignment and supervision module

This is the core operational component of Vital Studio.



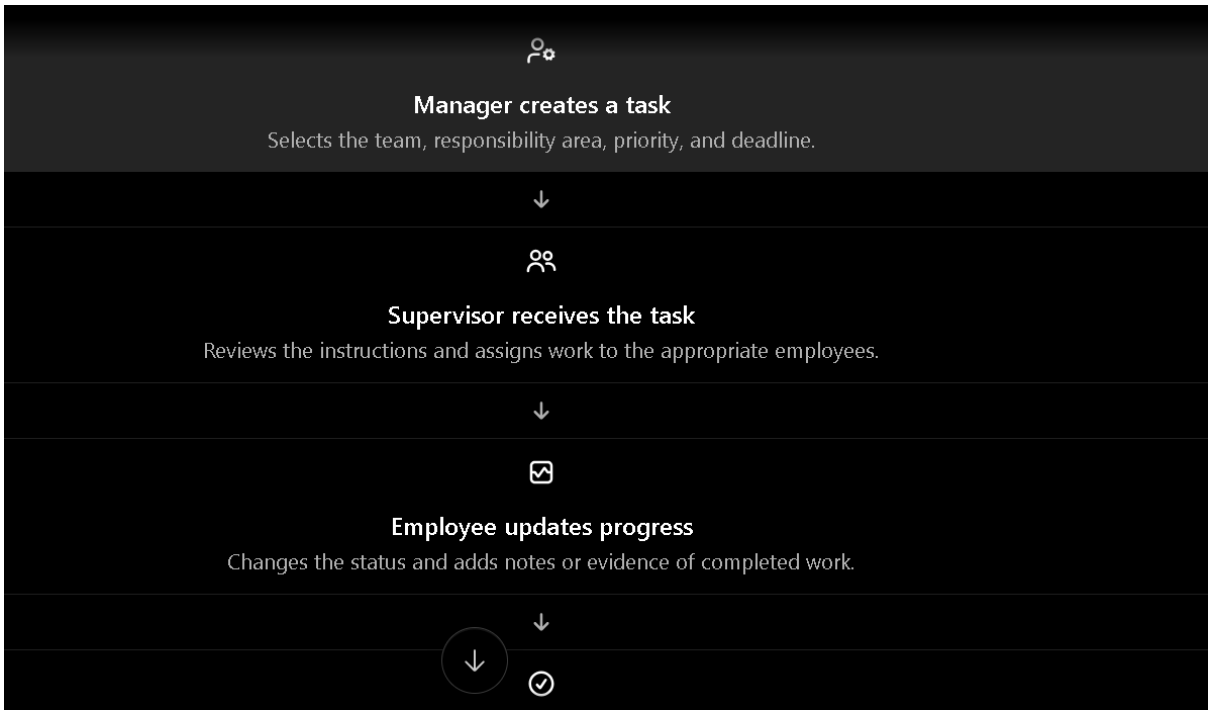
The manager should be able to create a task and assign it to a supervisor or an employee, depending on the level of responsibility.

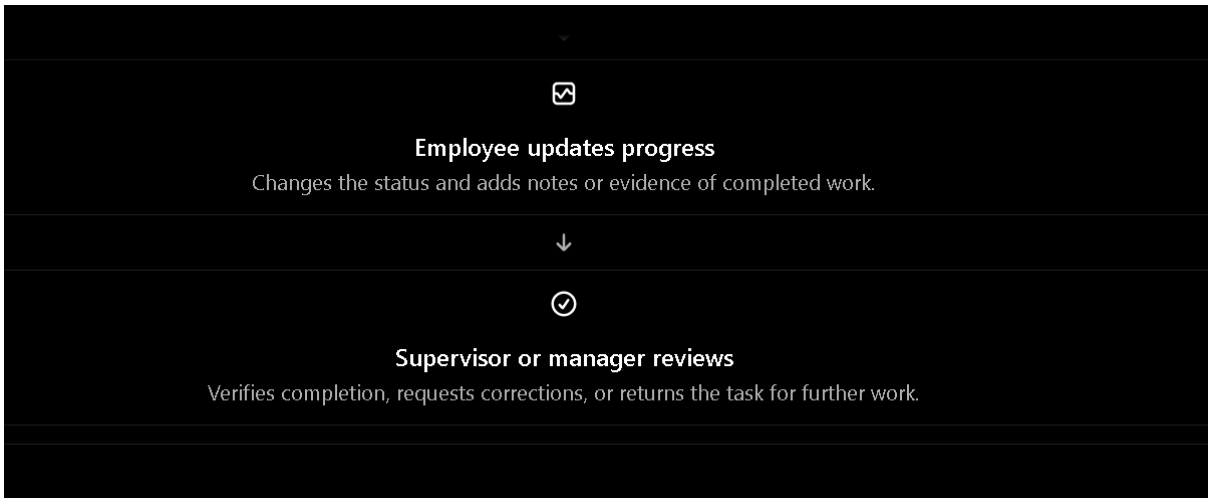
Each task should contain:

- Task title and description.
- Team or department.
- Area of responsibility.
- Assigned supervisor.
- Assigned employee, where applicable.

- Priority: Low, Medium, High, or Urgent.
- Creation date and deadline.
- Current status.
- Attachments or supporting documents, if needed.
- Comments, progress updates, and completion notes.

Example workflow





Task status rules

Status	Meaning
New	Created but not yet acknowledged
Received	Assignee has acknowledged the task
In Progress	Work has started
Completed	Work has been finished and accepted
Incomplete	Work could not be finished or was returned for correction
Deleted	Task has been removed from active work

For reliable reporting, keep a history of status changes, including who changed the status and when. Treat deleted tasks as archived or soft-deleted records rather than permanently erasing them, so the manager can audit previous activity.

5. Manager-to-staff communication center

The communication page should function as an internal announcement and messaging system.

Recommended features:

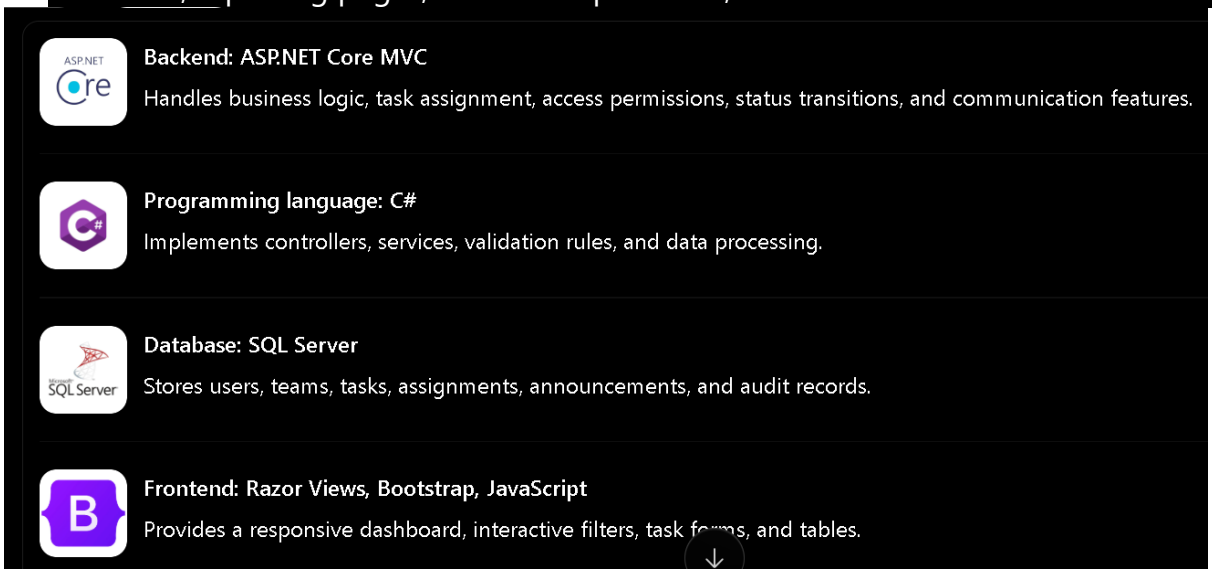
- **Create announcements:** Publish administrative notices, policies, instructions, and work updates.
- **Select recipients:** Send messages to all staff, specific teams, supervisors, or individual employees.
- **Set priority:** Normal, Important, or Urgent.
- **Track acknowledgments:** See who has read or acknowledged important instructions.
- **Maintain message history:** Search and filter previous announcements.
- **Task notifications:** Notify staff when a task is assigned, updated, approaching its deadline, or completed.

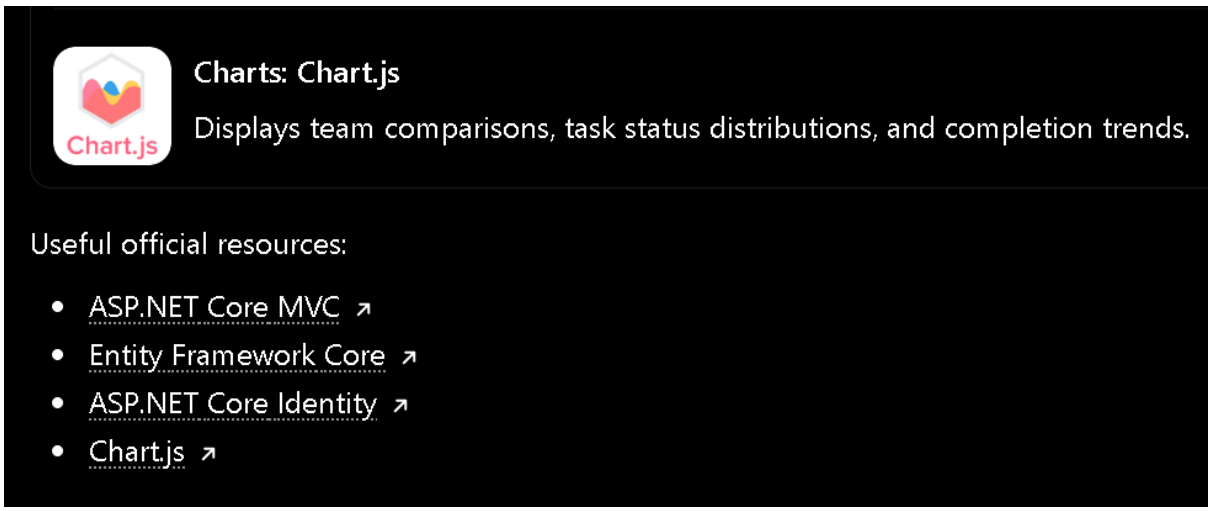
- **Attachments:** Share relevant documents when required.

- A useful distinction is between an announcement, which is broadcast to a group, and a direct message, which is intended for a particular person. If the initial project is small, start with announcements and task notifications, then add full direct messaging later.

6. Recommended technology for Vital Studio

- For this project, I recommend **ASP.NET Core MVC with C# and Entity Framework Core**. It is well suited to a role-based management system with forms, reporting pages, database operations, and administrative workflows.





Useful official resources:

- ASP.NET Core MVC
- Entity Framework Core
- ASP.NET Core Identity

- Chart.js

7. Database structure

A good database design will make the dashboard easier to maintain and expand.

Table	**Purpose**
Users	Managers, supervisors, and employees
Teams	Team names and descriptions
TeamMembers	Connects employees and supervisors to teams
Tasks	Task title, description, status, priority, and deadline
TaskAssignments	Tracks assigned users and assignment history
TaskStatusHistory	Records status changes and timestamps
Announcements	Messages published by management
AnnouncementRecipients	Tracks intended recipients and acknowledgments
TaskComments	Progress updates and discussions
AuditLogs	Records important administrative actions

A task should reference its team and its creator. Assignments should identify the responsible supervisor or employee. If a task passes from a supervisor to an employee, you can preserve that assignment history instead of overwriting the previous assignment.

8. Suggested ASP.NET Core project structure

text

VitalStudio/

├── Controllers/

│ ├── DashboardController.cs

```
│   ├── StaffController.cs
│   ├── TeamsController.cs
│   ├── TasksController.cs
│   ├── ReportsController.cs
│   └── CommunicationsController.cs
├── Models/
│   ├── ApplicationUser.cs
│   ├── Team.cs
│   ├── TaskItem.cs
│   ├── TaskAssignment.cs
│   └── Announcement.cs
├── Data/
│   └── ApplicationDbContext.cs
├── Services/
│   ├── DashboardService.cs
│   ├── TaskService.cs
│   └── NotificationService.cs
├── ViewModels/
│   └── ManagerDashboardViewModel.cs
├── Views/
│   ├── Dashboard/
│   ├── Staff/
│   ├── Teams/
│   ├── Tasks/
│   ├── Reports/
│   └── Communications/
└── wwwroot/
```

```
├── css/
├── js/
└── lib/
```

Keep the database queries and business rules in services or appropriate application layers rather than putting all the logic inside dashboard controllers. This will make the system easier to test and extend.

9. Example of a C# dashboard model

The following is a simplified example of the data that the manager's dashboard could receive from the backend.

csharp

```
public class ManagerDashboardViewModel
{
    public int TotalStaff { get; set; }
    public int TotalTasks { get; set; }
    public int NewTasks { get; set; }
    public int ReceivedTasks { get; set; }
    public int InProgressTasks { get; set; }
    public int CompletedTasks { get; set; }
    public int IncompleteTasks { get; set; }
    public int OverdueTasks { get; set; }

    public double CompletionRate { get; set; }

    public List<TeamPerformanceViewModel> TeamPerformance
        { get; set; } = new();

    public List<TaskSummaryViewModel> RecentTasks
        { get; set; } = new();
```

```
}


public class TeamPerformanceViewModel
{
    public string TeamName { get; set; } = "";
    public int AssignedTasks { get; set; }
    public int CompletedTasks { get; set; }
    public int InProgressTasks { get; set; }
    public int IncompleteTasks { get; set; }
}


public class TaskSummaryViewModel
{
    public int Id { get; set; }
    public string Title { get; set; } = "";
    public string TeamName { get; set; } = "";
    public string AssignedTo { get; set; } = "";
    public string Status { get; set; } = "";
    public DateTime? DueDate { get; set; }
}
```

Your DashboardService would calculate these values from the database, and DashboardController would pass the resulting model to the Razor view. Chart.js would then visualize the team performance and status data.

This is an illustrative model; the final implementation should use your actual entity relationships, workflow rules, and database schema.

10. How to update your existing dashboard

I recommend implementing the project in stages rather than rebuilding everything at once.

1. **Review the existing project.** Identify whether it uses ASP.NET Core MVC, Razor Pages, Blazor, or another frontend, and inspect its current database tables and authentication system.
2. **Improve the manager overview.** Add KPI cards, task-status charts, team-performance charts, and date filters using real database data.
3. **Build the task management module.** Implement task creation, team selection, supervisor assignment, deadlines, priorities, status changes, and task history.
4. **Add staff and team management.** Give managers visibility into team membership, individual workloads, and task ownership.
5. **Build the communication center.** Add announcements, recipient selection, read acknowledgments, and notifications.
6. **Apply security and permissions.** Managers should have oversight across authorized teams; supervisors should manage their assigned teams; employees should access their own tasks and relevant communications.
7. **Test and deploy.** Validate dashboard calculations, permissions, task transitions, mobile responsiveness, and reporting accuracy.

11. Let's tailor this to your actual project

I can help you design the updated dashboard and implement it in C#, including the database, controllers, Razor views, charts, and task assignment workflow.

The first thing I need to know is: **What technology does your existing Vital Studio dashboard use?**



2. Database

- (•) Microsoft SQL Server
- ( ) MySQL
- ( ) PostgreSQL
- ( ) Another database / not sure

3. Project status

- (•) I already have a dashboard and want to improve it
- ( ) I have some code, but the dashboard is incomplete
- ( ) I want to build the project from scratch

4. What should we implement first?

- (•) Manager dashboard, charts, and statistics
- ( ) Task assignment and status tracking
- ( ) Messaging and announcements
- ( ) The complete system, step by step